

Ghi Chú Phản Biện: Công Thức và Hình Ảnh trong Thesis

Portrait Artwork Reconstruction Framework (PARF)

Ngày tạo: 04/05/2026

Mục đích tài liệu

Tài liệu này là bản ghi chú riêng để phục vụ phần phản biện thesis. Mục tiêu của tài liệu không phải lặp lại toàn bộ nội dung báo cáo, mà là giúp giải thích rõ ba điểm quan trọng: công thức nào cần giữ trong report và vì sao cần giữ; công thức đó bám vào phần nào của code; và các hình ở Chapter 3, Chapter 4 thực sự đang chứng minh điều gì trong methodology và experiment. Tài liệu này cũng nêu rõ một số công thức đã được rút xuống thành mô tả bằng chữ để giảm rủi ro khi bị hỏi quá sâu vào những phần không bám 1-1 với implementation cuối cùng.

Nguyên tắc phản biện

Khi thầy hỏi về công thức, nên tách câu trả lời thành bốn lớp rất rõ. Lớp thứ nhất là công thức đó nằm ở trang nào và thuộc mục nào trong report. Lớp thứ hai là ý nghĩa của công thức trong pipeline. Lớp thứ ba là phần code tương ứng. Lớp thứ tư là mức độ bám sát implementation: có công thức là biểu diễn trực tiếp từ code, nhưng cũng có công thức là biểu diễn khái quát để giải thích cơ chế. Nếu nói rõ ngay điều này thì phần trả lời sẽ chắc hơn và ít bị bắt bẻ hơn.

1 Bản đồ công thức nên giữ

Công thức	Vị trí trong report	Vai trò	Code tương ứng	Mức độ bám code
Stage 1 input concat	Chương 3, Mục 3.2, trang 37	Tạo đầu vào cho coarse restoration	<code>networks/mat.py</code> , dòng 818	Trực tiếp
Stage 1 coarse output reinsertion	Chương 3, Mục 3.2, trang 37	Giữ nguyên pixel known, chỉ phục hồi vùng mask	<code>networks/mat.py</code> , dòng 855	Trực tiếp
Mask-aware attention	Chương 3, Mục 3.3, khoảng trang 40	Giải thích reasoning của transformer với token valid/invalid	<code>networks/mat.py</code> , dòng 186–220	Khái quát đúng ý

Công thức	Vị trí trong report	Vai trò	Code tương ứng	Mức độ bám code
Deterministic latent gate	Chương 3, Mục 3.4, trang 42	Trộn feature và latent guidance theo cách có điều khiển	<code>networks/mat.py</code> , dòng 137–138	Trực tiếp
Stage 2 input formation	Chương 3, Mục 3.5.1, trang 44	Tạo đầu vào cho refinement stage từ coarse result và visible pixels	<code>networks/mat.py</code> , dòng 909–910	Trực tiếp
Final reconstruction	Chương 3, Mục 3.5.2, trang 45	Ghép đầu ra cuối với vùng known của ảnh gốc	<code>networks/mat.py</code> , dòng 927	Trực tiếp
Total loss	Chương 3, Mục 3.6.1, trang 47	Tổng hợp loss của toàn pipeline	<code>losses/loss.py</code> , dòng 118	Trực tiếp
Adversarial loss	Chương 3, Mục 3.6.1, trang 47–48	Ép output nhìn giống ảnh thật ở cả Stage 1 và Stage 2	<code>losses/loss.py</code> , dòng 102–104, 148, 173	Trực tiếp
Perceptual loss	Chương 3, Mục 3.6.1, trang 48	So sánh trong không gian đặc trưng VGG	<code>losses/pcp.py</code> , dòng 91; <code>losses/loss.py</code> , dòng 41	Trực tiếp
FFL abstract loss	Chương 3, Mục 3.6.1, trang 49	Giải thích phần loss trong miền tần số	<code>losses/focal_frequency_loss.py</code> , dòng 43–53	Khá sát đúng ý
FFL warmup	Chương 3, Mục 3.6.2, trang 49	Tăng dần ảnh hưởng của FFL trong training	<code>training/schedule_utils.py</code> , dòng 48–53	Trực tiếp
PSNR, SSIM, LPIPS	Chương 4, Mục 4.5.1, trang 64	Bộ metric đánh giá định lượng	<code>evaluator/cal_psnr.py</code> , <code>evaluator/cal_ssim.py</code> , <code>evaluator/cal_lpips.py</code>	Trực tiếp

2 Các công thức đã rút xuống thành chữ

Ba nhóm công thức đã được chuyển từ equation sang mô tả bằng lời để report gọn hơn và an toàn hơn khi phản biện. Nhóm thứ nhất là công thức mô tả tiến trình nhiều stage như $T_1 \rightarrow T_5$. Nhóm này hữu ích để minh họa kiến trúc, nhưng không cần thiết phải viết thành equation vì người đọc đã có Figure 3.3 và Figure 3.1 để nhìn luồng. Nhóm thứ hai là công thức residual adapter. Trong report hiện tại, phần này chỉ nên trình bày như một ý tưởng tăng năng lực thích nghi ở tầng sâu, không nên đẩy thành một module toán học chi tiết nếu không muốn bị hỏi xoáy vào file class cụ thể. Nhóm thứ ba là cụm công thức SMM gốc của MAT như s_u , s_c , hay modulation weight W' , W'' . Những công thức này đúng theo paper nguồn, nhưng implementation thesis cuối cùng cần nhấn mạnh hơn vào deterministic latent gate và bottleneck fusion, nên việc rút chúng xuống thành mô

tả bằng lời là hợp lý hơn.

3 Phân tích chi tiết từng công thức

1. Stage 1 input concat

Vị trí trong report: Mục 3.2, khoảng trang 37.

Công thức:

$$\mathbf{F}_{\text{in}} = \text{Concat}(\mathbf{M} - 0.5, \mathbf{I} \odot \mathbf{M})$$

Ý nghĩa: Đây là đầu vào của Stage 1. Thành phần thứ nhất là mask được center về quanh 0 để mạng phân biệt rõ vùng known và unknown. Thành phần thứ hai là phần ảnh còn nhìn thấy được sau khi nhân với mask. Ý chính cần nói khi phản biện là Stage 1 không nhận ảnh gốc đầy đủ, mà nhận một biểu diễn đã chỉ rõ vùng thiếu và vùng còn lại.

Phân tích từng thành phần:

- $\mathbf{F}_{\text{in}}$ là tensor đầu vào mà Stage 1 thực sự nhận. Nó không còn là ảnh RGB thuần túy, mà là biểu diễn đã gắn thêm thông tin mask.
- $\text{Concat}(\cdot)$ cho biết hai nhóm thông tin được ghép theo chiều kênh. Ý nghĩa là model nhìn đồng thời trạng thái hỏng và nội dung còn quan sát được.
- $\mathbf{M}$ là binary mask. Trong ngữ cảnh code này, nó đóng vai trò chỉ ra vùng nào còn known và vùng nào cần khôi phục.
- $\mathbf{M} - 0.5$ là bước center hóa mask. Nếu mask gốc mang giá trị 0/1 thì sau khi trừ 0.5, nó thành $-0.5/0.5$. Ý nghĩa là đưa kênh mask về quanh 0 để dễ học hơn.
- $\mathbf{I}$ là ảnh input bị che. Đây không phải reference image đầy đủ, mà là ảnh thực tế mà model nhận khi infer.
- $\odot$ là nhân theo từng pixel. Ý nghĩa là chỉ giữ lại phần ảnh còn quan sát được theo logic mask.
- $\mathbf{I} \odot \mathbf{M}$ là visible content. Thành phần này cung cấp chứng cứ ảnh thật cho Stage 1 trước khi mô hình phải suy luận phần bị che.

Code tương ứng: `networks/mat.py`, dòng 818: `x = torch.cat([masks_in - 0.5, images_in * masks_in], dim=1)`.

Điểm nên nhấn mạnh: Công thức này bám trực tiếp vào implementation. Nếu bị hỏi “vì sao trừ 0.5”, có thể trả lời là để đưa mask nhị phân về dạng centered input, giúp mạng không luôn nhìn mask như một kênh chỉ toàn giá trị dương.

2. Stage 1 coarse output reinsertion

Vị trí trong report: Mục 3.2, khoảng trang 37.

Công thức:

$$\hat{\mathbf{I}}^{(1)} = \hat{\mathbf{I}}_{\text{raw}}^{(1)} \odot (1 - \mathbf{M}) + \mathbf{I} \odot \mathbf{M}$$

Ý nghĩa: Stage 1 sinh ra ảnh coarse, nhưng không được phép làm hỏng vùng ảnh đã biết. Vì vậy phần ảnh known được chèn lại từ ảnh đầu vào. Đây là một công thức rất đáng giữ vì nó thể hiện rõ triết lý non-invasive restoration trong code.

Phân tích từng thành phần:

- $\hat{\mathbf{I}}^{(1)}$ là coarse output sau khi đã ghép lại với vùng known. Đây mới là ảnh được chuyển sang Stage 2.
- $\hat{\mathbf{I}}_{\text{raw}}^{(1)}$ là output thô do decoder của Stage 1 sinh ra. Nó có thể ảnh hưởng cả vùng known nếu không có bước ghép lại.
- $(1 - \mathbf{M})$ là binary mask đảo. Nó chọn ra vùng cần khôi phục, tức là vùng unknown.
- $\hat{\mathbf{I}}_{\text{raw}}^{(1)} \odot (1 - \mathbf{M})$ có nghĩa là chỉ lấy phần model được phép sinh ra ở vùng hỏng.
- $\mathbf{I} \odot \mathbf{M}$ là visible known pixels từ input. Thành phần này chèn lại thông tin gốc.
- Dấu cộng ở đây không phải cộng pha trộn mơ hồ. Hai vế tạo thành hai vùng bù trừ nhau nhờ mask và inverse mask.

Code tương ứng: `networks/mat.py`, dòng 855.

Điểm nên nhấn mạnh: Mạng chỉ chịu trách nhiệm cho vùng bị khuyết. Vùng quan sát được luôn được bảo toàn.

3. Mask-aware attention

Vị trí trong report: Mục 3.3, khoảng trang 40.

Công thức:

$$\text{Att}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T + \mathbf{M}'}{\sqrt{d_k}} \right) \mathbf{V}$$

Ý nghĩa: Đây là công thức nền để giải thích vì sao transformer trong PARF không xử lý mọi token như nhau. Các token invalid bị chặn hoặc bị giảm mạnh ảnh hưởng thông qua bias mask.

Phân tích từng thành phần:

- $\text{Att}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ là toàn bộ phép tính attention. Nó nhận query, key, value và trả về feature đã tái phân bố thông tin.
- $\mathbf{Q}$ là query. Về ý nghĩa, nó đại diện cho “token hiện tại đang hỏi thông tin từ ai”.
- $\mathbf{K}$ là key. Nó đại diện cho “mỗi token khác có thể được tham chiếu như thế nào”.
- $\mathbf{V}$ là value. Đây là nội dung thực sự sẽ được lấy rồi trộn về.
- $\mathbf{Q}\mathbf{K}^T$ là ma trận điểm tương quan giữa token đang xét và các token tham chiếu. Điểm càng cao thì token đó càng có ảnh hưởng.
- $\mathbf{M}'$ là mask bias trong biểu diễn khái quát. Nó làm giảm hoặc chặn sự tương tác với token không hợp lệ.

- $\sqrt{d_k}$ là hệ số chuẩn hóa theo chiều key. Mục đích là giữ độ lớn logits attention ổn định hơn.
- $\text{Softmax}(\cdot)$ biến điểm tương quan thành trọng số chuẩn hóa. Tổng trọng số trên mỗi hàng bằng 1.
- Sản phẩm cuối cùng với $\mathbf{V}$ có nghĩa là attention lấy tổ hợp có trọng số của các value vectors.

Code tương ứng: `networks/mat.py`, từ khoảng dòng 186 đến 220. Ở đó attention còn có thêm các chi tiết implementation như normalization, relative bias, shifted-window mask và key bias -100 .

Điểm nên nhấn mạnh: Đây là công thức khái quát đúng cơ chế, chứ không phải bản sao line-by-line của code. Nếu bị hỏi sâu, nên nói thẳng rằng report viết ở mức conceptual formulation, còn implementation có thêm các thành phần ổn định hóa attention.

4. Deterministic latent gate

Vị trí trong report: Mục 3.4, trang 42.

Công thức:

$$\mathbf{g} = \sigma(f_\theta([\mathbf{f}, \mathbf{l}))), \quad \tilde{\mathbf{f}} = \mathbf{g} \odot \mathbf{f} + (1 - \mathbf{g}) \odot \mathbf{l}$$

Ý nghĩa: Đây là cơ chế thesis cần nhấn mạnh nhất ở phần style guidance. Nó không trộn feature và latent theo tỉ lệ cố định, mà học ra gate để quyết định lúc nào nên tin feature cấu trúc, lúc nào nên dùng thêm latent guidance.

Phân tích từng thành phần:

- $\mathbf{f}$ là feature map đang có từ nhánh cấu trúc/chính. Nó mang thông tin không gian mà encoder-decoder đã suy ra.
- $\mathbf{l}$ là latent guidance từ style branch. Nó mang thông tin bổ sung về phong cách và xu hướng hoàn thiện.
- $[\mathbf{f}, \mathbf{l}]$ là thao tác ghép hai nguồn thông tin. Ý nghĩa là gate không quyết định chỉ dựa trên một phía.
- $f_\theta(\cdot)$ là hàm học được, trong code là một cụm conv nhỏ để sinh gate map.
- $\sigma(\cdot)$ là sigmoid. Nó ép giá trị gate về khoảng $(0, 1)$ để có thể hiểu như tỉ lệ trộn mềm.
- $\mathbf{g}$ là gate map. Mỗi vị trí không gian và mỗi kênh có thể có trọng số riêng.
- $\mathbf{g} \odot \mathbf{f}$ là phần đóng góp của feature chính khi gate cao.
- $(1 - \mathbf{g}) \odot \mathbf{l}$ là phần đóng góp của latent guidance khi gate thấp hơn.
- $\tilde{\mathbf{f}}$ là feature sau khi trộn. Đây là feature thực sự đi tiếp vào các bước sau.

Code tương ứng: `networks/mat.py`, dòng 137–138. Phần gọi thực tế xuất hiện ở dòng 845 và 916 qua hàm `blend_with_latent_gate`.

Điểm nên nhấn mạnh: Đây là công thức bám rất sát code. Nếu thầy hỏi contribution của PARF ở phần style là gì, nên trả lời rằng điểm thesis-developed rõ nhất là deterministic gating, không phải toàn bộ style branch gốc của MAT.

5. Stage 2 input formation

Vị trí trong report: Mục 3.5.1, trang 44.

Công thức:

$$\mathbf{I}_{\text{ref}} = \mathbf{I} \odot \mathbf{M} + \hat{\mathbf{I}}^{(1)} \odot (1 - \mathbf{M}), \quad \mathbf{F}_{\text{ref}} = \text{Concat}(\mathbf{M} - 0.5, \mathbf{I}_{\text{ref}}, \mathbf{I} \odot \mathbf{M})$$

Ý nghĩa: Đây là chỗ chứng minh Stage 2 không làm việc từ đầu. Nó nhận coarse result đã ghép lại với visible pixels, sau đó concat thêm mask và visible content để refinement có đủ ngữ cảnh.

Phân tích từng thành phần:

- $\mathbf{I}_{\text{ref}}$ là ảnh trung gian dùng để đưa vào Stage 2. Nó đã có coarse completion ở vùng thiếu và known pixels ở vùng quan sát được.
- $\mathbf{I}$ là masked input ban đầu.
- $\mathbf{M}$ là binary mask, tiếp tục đóng vai trò phân vùng known/unknown.
- $\hat{\mathbf{I}}^{(1)}$ là coarse output từ Stage 1.
- $\mathbf{I} \odot \mathbf{M}$ giữ nguyên visible pixels từ input. Thành phần này đảm bảo Stage 2 vẫn nhìn thấy bằng chứng thật.
- $\hat{\mathbf{I}}^{(1)} \odot (1 - \mathbf{M})$ đưa phần ước lượng coarse vào đúng vùng cần khôi phục.
- $\mathbf{F}_{\text{ref}}$ là tensor đầu vào thật của refinement encoder.
- $\text{Concat}(\mathbf{M} - 0.5, \mathbf{I}_{\text{ref}}, \mathbf{I} \odot \mathbf{M})$ cho thấy Stage 2 vừa biết mask, vừa biết coarse composite, vừa biết vùng nhìn thấy thật. Đây là lý do nó không chỉ làm mịn mù quáng.

Code tương ứng: networks/mat.py, dòng 909–910.

Điểm nên nhấn mạnh: Đây là khác biệt lớn so với một pipeline single-stage. Stage 2 là refinement dựa trên coarse prior, không phải sinh lại toàn bộ khuôn mặt từ đầu.

6. Final reconstruction

Vị trí trong report: Mục 3.5.2, trang 45.

Công thức:

$$\hat{\mathbf{I}} = \hat{\mathbf{I}}^{(2)} \odot (1 - \mathbf{M}) + \mathbf{I} \odot \mathbf{M}$$

Ý nghĩa: Tương tự Stage 1, đầu ra cuối vẫn phải giữ nguyên vùng known. Công thức này thể hiện nguyên tắc bảo toàn thông tin gốc.

Phân tích từng thành phần:

- $\hat{\mathbf{I}}$ là final restored output mà thesis báo cáo.
- $\hat{\mathbf{I}}^{(2)}$ là đầu ra thuần từ refinement decoder trước khi ghép lại.
- $(1 - \mathbf{M})$ tiếp tục chọn vùng cần phục hồi.

- $\mathbf{I} \odot \mathbf{M}$ bảo toàn vùng đã biết từ input ban đầu.
- Cấu trúc “phần sinh ra” cộng với “phần giữ lại” cho thấy ảnh cuối là composite có kiểm soát, không phải ảnh phát sinh lại toàn bộ.

Code tương ứng: `networks/mat.py`, dòng 927.

Điểm nên nhấn mạnh: Final result là refinement trên vùng khuyết, không phải thay đổi toàn ảnh.

7. Total loss

Vị trí trong report: Mục 3.6.1, trang 47.

Công thức:

$$\mathcal{L}_G^{\text{total}} = \mathcal{L}_G^{(2)} + \mathcal{L}_G^{(1)} + \lambda_p \mathcal{L}_P + \lambda_f \mathcal{L}_{\text{FFL}}$$

Ý nghĩa: Đây là công thức rất quan trọng khi bảo vệ. Nó cho thấy training objective không chỉ tối ưu output cuối, mà còn ràng buộc Stage 1. Đồng thời, loss không dựa vào một tiêu chí duy nhất mà phối hợp realism, semantics và frequency detail.

Phân tích từng thành phần:

- $\mathcal{L}_G^{\text{total}}$ là loss cuối mà generator tối ưu ở mỗi bước train.
- $\mathcal{L}_G^{(2)}$ là adversarial generator loss cho final output. Nó ép ảnh cuối nhìn thật hơn.
- $\mathcal{L}_G^{(1)}$ là adversarial generator loss cho coarse output. Nó giữ cho Stage 1 không quá yếu.
- λ_p là hệ số cân bằng cho perceptual loss. Nó quyết định perceptual term mạnh đến đâu so với các term còn lại.
- $\mathcal{L}_P$ là perceptual loss trong không gian đặc trưng.
- λ_f là trọng số danh nghĩa của FFL.
- $\mathcal{L}_{\text{FFL}}$ là frequency-domain loss. Nó nhấn vào khác biệt về phổ tần số giữa output và reference.
- Cấu trúc cộng tuyến tính cho thấy đây là multi-objective training được đơn giản hóa thành một hàm mất mát tổng.

Code tương ứng: `losses/loss.py`, dòng 118.

Điểm nên nhấn mạnh: Công thức này là tổng hợp trung thực của implementation training.

8. Adversarial loss

Vị trí trong report: Mục 3.6.1, trang 47–48.

Công thức chính:

$$\mathcal{L}_G^{(2)} = \mathbb{E}[\text{softplus}(-D(\hat{\mathbf{I}}))], \quad \mathcal{L}_G^{(1)} = \mathbb{E}[\text{softplus}(-D^{(1)}(\hat{\mathbf{I}}^{(1)}))]$$

Ý nghĩa: Adversarial loss trong thesis này không chỉ dùng cho final output, mà còn giám sát coarse output. Đó là điểm đáng nhấn mạnh khi bị hỏi vì sao Stage 1 tốt hơn một intermediate image bình thường.

Phân tích từng thành phần:

- $\mathcal{L}_G^{(2)}$ là loss generator tương ứng với đầu ra cuối.
- $D(\hat{\mathbf{I}})$ là điểm mà discriminator gán cho final output. Nếu discriminator cho điểm thấp theo nghĩa “ảnh giả”, generator sẽ bị phạt nhiều hơn.
- Dấu âm trong $-D(\hat{\mathbf{I}})$ phản ánh generator muốn đẩy score theo hướng “real”.
- $\text{softplus}(\cdot)$ là dạng mất mát trơn thường dùng trong GAN implementation kiểu logistic.
- $\mathbb{E}[\cdot]$ biểu diễn kỳ vọng trên minibatch. Nghĩa thực tế là lấy trung bình theo mẫu trong một batch train.
- $\mathcal{L}_G^{(1)}$ có vai trò tương tự nhưng áp cho coarse output.
- $D^{(1)}(\hat{\mathbf{I}}^{(1)})$ là đánh giá của discriminator nhánh Stage 1 hoặc đánh giá đặt trên coarse result trong cấu hình loss hiện tại.

Code tương ứng: `losses/loss.py`, dòng 102–104. Discriminator fake/real ở dòng 148 và 173. R1 ở dòng 185 và 190.

Điểm nên nhấn mạnh: Stage 1 bị giám sát để coarse layout có chất lượng thực sự, nhờ đó Stage 2 có nền tốt hơn để refine.

9. Perceptual loss

Vị trí trong report: Mục 3.6.1, trang 48.

Công thức:

$$\mathcal{L}_P = \sum_i \eta_i \left\| \phi_i(\hat{\mathbf{I}}) - \phi_i(\mathbf{I}) \right\|_1$$

Ý nghĩa: Đây là loss trong không gian đặc trưng VGG, giúp model quan tâm đến tương đồng cảm nhận thay vì chỉ khác biệt pixel. Với portrait artwork, điều này quan trọng vì mắt, mũi, miệng có thể đúng cảm nhận dù không khớp từng pixel.

Phân tích từng thành phần:

- $\mathcal{L}_P$ là perceptual loss tổng.
- $\sum_i$ cho thấy loss này cộng đóng góp từ nhiều tầng đặc trưng.
- η_i là trọng số của tầng thứ i . Tầng nào quan trọng hơn có thể được đặt trọng số lớn hơn.
- $\phi_i(\cdot)$ là hàm trích đặc trưng tại layer i của mạng VGG đã pretrained.
- $\phi_i(\hat{\mathbf{I}})$ là đặc trưng của ảnh phục hồi.
- $\phi_i(\mathbf{I})$ là đặc trưng của ảnh tham chiếu gốc.

- Chuẩn $\|\cdot\|_1$ đo chênh lệch tuyệt đối giữa hai đặc trưng. Nó thường ổn định và ít làm mờ hơn một số lựa chọn khác.

Code tương ứng: `losses/loss.py`, dòng 41 khởi tạo `conv4_4` và `conv5_4`; `losses/pcp.py`, dòng 91 cộng loss theo layer.

Điểm nên nhấn mạnh: Thesis không phát minh perceptual loss. Contribution là phối hợp nó đúng chỗ trong two-stage training.

10. FFL abstract loss

Vị trí trong report: Mục 3.6.1, trang 49.

Công thức:

$$\mathcal{L}_{\text{FFL}} = \mathcal{F}_{\text{freq}}(\hat{\mathbf{I}}, \mathbf{I})$$

Ý nghĩa: Report cố ý giữ công thức FFL ở mức abstract. Điều này hợp lý vì implementation thực tế còn có normalize từ $[-1, 1]$ về $[0, 1]$, FFT2, trọng số frequency-adaptive và mean reduction.

Phân tích từng thành phần:

- $\mathcal{L}_{\text{FFL}}$ là giá trị loss trong miền tần số.
- $\mathcal{F}_{\text{freq}}(\cdot, \cdot)$ không phải một phép toán đơn lẻ, mà là ký hiệu gộp cho chuỗi bước: chuẩn hóa ảnh, biến đổi Fourier, tính sai khác phổ, gán trọng số theo độ khó, rồi lấy trung bình.
- $\hat{\mathbf{I}}$ là ảnh phục hồi cần đánh giá.
- $\mathbf{I}$ là ảnh tham chiếu gốc.
- Ý nghĩa sâu hơn là loss này không nhìn trực tiếp pixel, mà nhìn xem cấu trúc tần số của ảnh phục hồi có gần ảnh gốc hay không.

Code tương ứng: `losses/focal_frequency_loss.py`, dòng 43–53.

Điểm nên nhấn mạnh: Đây là công thức khái quát để giải thích mục tiêu. Nếu thầy hỏi sâu, hãy nói thêm rằng code dùng trọng số theo độ khó phổ tần số và chuẩn hóa theo maximum trong từng bản đồ weight.

11. FFL warmup

Vị trí trong report: Mục 3.6.2, trang 49.

Công thức:

$$\lambda_f^{\text{eff}} = \lambda_f \cdot \min\left(1, \frac{k_{\text{img}}}{k_{\text{warm}}}\right)$$

Ý nghĩa: Đây là chiến lược ổn định training. Giai đoạn đầu model nên học cấu trúc coarse trước. Sau đó mới tăng dần ảnh hưởng của FFL để tránh làm học quá sớm vào texture frequency.

Phân tích từng thành phần:

- λ_f^{eff} là trọng số FFL thực tế đang được dùng ở thời điểm train hiện tại.
- λ_f là trọng số mục tiêu hoặc trọng số trần. Khi warmup kết thúc, trọng số hiệu dụng sẽ tiến về giá trị này.
- $\min\left(1, \frac{k_{\text{img}}}{k_{\text{warm}}}\right)$ là hệ số warmup.
- k_{img} là số lượng ảnh hoặc bước ảnh đã đi qua trong tiến trình training, tùy cách code đếm.
- k_{warm} là ngưỡng warmup. Trước ngưỡng này, FFL tăng dần; sau ngưỡng này, FFL giữ ổn định ở trọng số đầy đủ.
- Hàm min bảo đảm hệ số không vượt quá 1. Nghĩa là warmup chỉ tăng đến mức đầy đủ, không khuếch đại quá mức.

Code tương ứng: `training/schedule_utils.py`, dòng 48–53; được gọi từ `losses/loss.py`, dòng 55 và 95.

Điểm nên nhấn mạnh: Đây là một điểm thesis rất dễ bảo vệ vì vừa có lý do học thuật, vừa có code trực tiếp.

12. Metric formulas

Vị trí trong report: Mục 4.5.1, trang 64.

Ý nghĩa: Trong report hiện tại, metric được trình bày dưới dạng mô tả và bảng mục đích sử dụng. Tuy nhiên khi phản biện, bạn nên nắm công thức chuẩn của PSNR, SSIM, LPIPS vì code evaluation dùng trực tiếp các chuẩn đó.

Phân tích từng thành phần:

- Với PSNR, thành phần gốc là MSE. MSE đo sai khác trung bình theo pixel. Sau đó PSNR biến MSE sang thang logarit. MSE càng nhỏ thì PSNR càng cao.
- Với SSIM, ba nhóm thành phần chính là độ sáng trung bình, độ tương phản, và mức nhất quán cấu trúc. Ý nghĩa là SSIM không chỉ nhìn sai số từng pixel mà nhìn sự giống nhau về cấu trúc hình ảnh.
- Với LPIPS, hai ảnh được đưa qua mạng đặc trưng pretrained. Khoảng cách được đo trong không gian đặc trưng thay vì không gian pixel. LPIPS càng thấp thì cảm nhận thị giác càng gần hơn.

Code tương ứng:

- PSNR: `evaluator/cal_psnr_ssim_l1.py`, dòng 15 và 19.
- SSIM: `evaluator/cal_psnr_ssim_l1.py`, dòng 23 và 40.
- LPIPS: `evaluator/cal_lpips.py`, dòng 26 và 40.

Điểm nên nhấn mạnh: PSNR và SSIM trong code được tính trên full image với range 255. LPIPS dùng AlexNet và ảnh được normalize về khoảng $[-1, 1]$.

4 Phân tích chi tiết các hình ở Chapter 3

Figure 3.1 – Overall architecture of PARF for portrait artwork reconstruction (trang 35)

Đây là hình quan trọng nhất của Chapter 3. Nó cho người đọc cái nhìn tổng thể về PARF như một pipeline hai giai đoạn. Khi giải thích hình này, nên nói rõ luồng xử lý theo thứ tự: ảnh chân dung đã bị che và binary mask đi vào Stage 1; Stage 1 chịu trách nhiệm coarse restoration để khôi phục bố cục mặt ở mức tổng quát; sau đó kết quả coarse được đưa sang Stage 2 cùng với visible pixels và mask; Stage 2 refinement chịu trách nhiệm làm mượt biên, ổn định chi tiết mặt, và hòa hợp phong cách tranh. Phần style branch trong hình cho thấy thesis không chỉ phục hồi cấu trúc mà còn cố gắng giữ painterly appearance. Cuối cùng, block optimization nhấn mạnh rằng chất lượng đầu ra đến từ tổ hợp adversarial, perceptual, và FFL chứ không phải một loss đơn lẻ.

Về code, hình này tương ứng tổng hợp của `networks/mat.py`, `losses/loss.py`, `losses/focal_frequency` và `training/schedule_utils.py`. Nếu bị hỏi “hình này có đúng với code không”, câu trả lời an toàn là: đúng ở mức architecture overview; hình này gom những thành phần cốt lõi của forward path và training objective vào một sơ đồ chung.

Figure 3.2 – Illustration of the Stage 1 coarse restoration process (trang 38)

Hình này chỉ tập trung vào Stage 1. Khi giải thích, nên nhấn mạnh Stage 1 không có nhiệm vụ tạo ra ảnh cuối đẹp nhất. Nhiệm vụ của nó là ước lượng cấu trúc khuôn mặt thô, đặc biệt là vị trí tương đối của mắt, mũi, miệng và biên khuôn mặt khi vùng bị khuyết khá lớn. Đây là lý do report gọi nó là *coarse restoration*. Hình này cũng làm rõ đầu vào của Stage 1 luôn gồm masked portrait và binary mask, không phải ảnh đầy đủ.

Về code, hình này bám vào `FirstStage.forward` trong `networks/mat.py`, đặc biệt ở phần tạo đầu vào dòng 818, reasoning qua transformer, rồi reinsert visible pixels ở dòng 855. Khi phản biện, bạn nên nói rằng Stage 1 giúp Stage 2 không phải học vừa cấu trúc tổng quát vừa chi tiết cục bộ cùng một lúc.

Figure 3.3 – Structure of the transformer stage used in PARF (trang 40)

Hình này giải thích lõi reasoning của PARF. Ý chính cần nói là transformer stage cho phép trao đổi thông tin xa, nên vùng mặt bị khuyết có thể suy ra từ những vùng còn quan sát được ở xa hơn, chứ không chỉ từ vùng lân cận rất nhỏ như CNN thuần túy. Nếu bị hỏi vì sao hình này quan trọng, có thể trả lời rằng portrait restoration đặc biệt cần khả năng reasoning dài hạn vì một mắt có thể phải suy từ mắt còn lại, từ orientation của đầu, hoặc từ contour tổng thể.

Về code, hình này tương ứng với `WindowAttention`, `SwinTransformerBlock`, và `BasicLayer` trong `networks/mat.py`. Đây là nơi report dùng công thức attention như một mô tả khái quát.

Figure 3.4 – Illustration of the dynamic mask updating strategy during transformer reasoning (trang 42)

Hình này mô tả cách valid region được mở rộng dần trong quá trình reasoning. Thay vì xem toàn bộ vùng bị che là không thể dùng trong suốt forward pass, transformer cập nhật dần thông tin hợp lệ khi đã suy được ngữ cảnh đáng tin cậy hơn. Với portrait artwork, điều này rất hữu ích vì mô hình có thể dần lan truyền ngữ cảnh từ vùng còn tốt vào vùng mặt bị hỏng.

Về code, phần này liên quan đến xử lý `mask_windows` trong `WindowAttention.forward`. Nếu thầy hỏi tại sao cần hình này, nên trả lời rằng nó trực quan hóa vì sao PARF có thể xử lý large-hole mask tốt hơn một cách tiếp cận chỉ nhìn local patch cố định.

Figure 3.5 – Style-code generation path for the Style Manipulation Module in PARF (trang 43)

Hình này không nhằm chứng minh một công thức toán cụ thể, mà nhằm tách riêng style branch ra khỏi main branch. Khi nói về hình này, bạn nên nhấn mạnh rằng restoration của tranh chân dung không chỉ cần đúng hình học, mà còn cần hòa hợp với tông màu, texture, và painterly continuity của phần tranh còn lại. Style branch đóng vai trò tạo latent/style guidance, sau đó guidance này được đưa vào pipeline thông qua deterministic latent gate và bottleneck fusion.

Về code, hình này liên quan đến `ws_style`, `to_square`, `to_style`, `DeterministicLatentGate`, và `blend_with_latent_gate` trong `networks/mat.py`. Điểm nên nhấn mạnh là thesis không claim phát minh toàn bộ style branch, mà tập trung vào cách tích hợp nó cho portrait-artwork refinement.

Figure 3.6 – Illustration of the Stage 2 refinement process in PARF (trang 45)

Đây là hình quan trọng thứ hai sau Figure 3.1. Hình này thể hiện rõ luận điểm “coarse first, refine later”. Khi giải thích, nên nói Stage 2 nhận coarse result đã có bố cục mặt tương đối ổn, rồi dùng refinement encoder, bottleneck fusion và decoder để xử lý những lỗi mà Stage 1 còn để lại: biên gượng, vùng mặt quá mềm, tone transition chưa ổn, hoặc facial part chưa đủ thuyết phục. Đây là cách report bảo vệ sự tồn tại của Stage 2 như một thành phần độc lập, không phải chỉ là hậu xử lý nhẹ.

Về code, hình này bám rất sát `SynthesisNet.forward` trong `networks/mat.py`, đặc biệt các dòng 909–927. Khi bị hỏi “khác gì single-stage baseline”, hãy nhấn mạnh rằng Stage 2 dùng coarse output như structural prior và tập trung vào local correction.

Figure 3.7 – Training objectives of PARF (trang 46)

Hình này giúp nối architecture với optimization. Nó cho thấy PARF không tối ưu theo một loss duy nhất, mà dùng một bộ ràng buộc bổ sung cho nhau. Adversarial loss giúp ảnh nhìn thật hơn, perceptual loss giúp giữ semantic similarity, và FFL giúp giữ

texture/painterly detail trong miền tần số. Khi giải thích, nên nói rõ hình này đặc biệt quan trọng vì thesis của bạn không nằm ở phát minh từng loss riêng lẻ, mà ở cách phối hợp chúng cho bài toán chân dung tranh.

Về code, hình này tương ứng trực tiếp với `losses/loss.py`, `losses/focal_frequency_loss.py`, và `training/schedule_utils.py`. Đây là hình rất dễ bảo vệ vì code map khá rõ.

5 Phân tích chi tiết các hình ở Chapter 4

Figure 4.1 – Representative artwork samples prepared for experimental evaluation (trang 55)

Hình này có vai trò mô tả dữ liệu đầu vào của thí nghiệm. Nó cho người đọc thấy các mẫu tranh chân dung đã được chọn để đánh giá có đặc điểm gì về góc mặt, màu sắc, độ mềm của nét vẽ, và độ đa dạng phong cách. Khi phản biện, nên nhấn mạnh rằng dữ liệu không phải ảnh tự nhiên mà là portrait artwork, nên bài toán khó ở chỗ vừa cần phục hồi cấu trúc mặt vừa phải tôn trọng phong cách hội họa.

Figure 4.2 – Prepared damage masks and masked artwork inputs used in the experiments (trang 57)

Hình này giải thích điều kiện hư hại mà PARF và các baseline phải đối mặt. Nó quan trọng vì nếu không cho thấy mask pattern, người đọc sẽ khó đánh giá các kết quả qualitative ở phía sau. Khi nói về hình này, nên nhấn mạnh mask được thiết kế để thường xuyên chồng lên vùng mặt quan trọng, vì mục tiêu thesis tập trung vào portrait restoration chứ không phải background completion thông thường.

Figure 4.3 – Qualitative restoration results of PARF (trang 59)

Hình này là bằng chứng qualitative đầu tiên rằng PARF hoạt động theo đúng logic hai giai đoạn. Mỗi hàng nên được đọc theo thứ tự: ảnh gốc, ảnh bị che, Stage 1 output, final output. Nếu bị hỏi “hình này chứng minh điều gì”, câu trả lời là: nó chứng minh Stage 1 đã tạo được facial layout đủ ổn, và Stage 2 giúp kết quả cuối hòa nhập hơn với phần tranh xung quanh. Hình này đặc biệt hữu ích để nói về cooperation giữa coarse structure recovery và local refinement.

Figure 4.4 – Qualitative comparison of PARF, MAT, and recent baselines (trang 63)

Đây là hình comparison quan trọng nhất của Chapter 4. Khi giải thích, nên nói rằng hình này đặt tất cả method dưới cùng một masked input để so sánh công bằng. Mỗi hàng thể hiện một kiểu hành vi khác nhau: có method quá mượt, có method sai cấu trúc, có method giữ texture nhưng lệch facial detail, và có method xuất hiện artifact. PARF được kỳ vọng thể hiện cân bằng hơn giữa facial plausibility, contour stability, và painterly

continuity. Điểm tốt của hình này là người đọc có thể thấy trực tiếp vì sao metric tốt chưa đủ, mà cần qualitative evidence.

Figure 4.5 – Visual checkpoint-based ablation comparison of the selected thesis checkpoints (trang 66)

Hình này không phải so sánh với baseline bên ngoài, mà so sánh các giai đoạn phát triển nội bộ của chính framework. Khi thuyết trình, nên nói rõ hàng đầu tiên là checkpoint sớm chỉ có frequency-aware training, hàng tiếp theo là Phase 1 architecture, và hàng cuối là Phase 2 architecture. Hình này giúp giải thích rằng cải thiện cuối cùng không đến từ một thay đổi duy nhất, mà đến từ cả training design lẫn architecture refinement. Nó là qualitative counterpart của bảng ablation metrics.

Figure 4.6 – Failure cases and remaining limitations of PARF (trang 67)

Hình này rất quan trọng khi phản biện vì nó thể hiện thái độ học thuật cân bằng. Thay vì chỉ trình bày case đẹp, report còn chỉ ra ranh giới mà framework hiện tại chưa xử lý tốt: khi vùng mask che gần hết khuôn mặt hoặc che hơn một nửa các thành phần chính. Khi giải thích, nên nhấn mạnh rằng framework vẫn có thể tạo ra face-like completion ở mức tổng quát, nhưng dễ sai ở alignment của mắt, cầu mũi, miệng, hoặc ở độ chính xác của cấu trúc cục bộ. Đây là chỗ tự nhiên để nối sang future work.

6 Cách trả lời ngắn khi bị hỏi nhanh

Nếu bị hỏi “công thức nào là trọng tâm nhất của thesis”, nên trả lời theo thứ tự: Stage 2 input formation, deterministic latent gate, total loss, và FFL warmup. Nếu bị hỏi “công thức nào là của paper gốc”, nên nói rõ mask-aware attention và adversarial/perceptual/FFL đều kế thừa từ literature, còn contribution của thesis nằm ở cách tổ chức chúng trong PARF. Nếu bị hỏi “phần nào trong report dễ bị hỏi xoáy nhất”, nên tự lưu ý rằng style branch và transformer attention là phần nên trả lời ở mức cơ chế khái quát, không nên tự đẩy sang mức chứng minh toán học quá sâu nếu thầy không yêu cầu.

Kết luận ngắn

Để phản biện chắc, không cần thuộc quá nhiều công thức. Chỉ cần nắm thật vững những công thức còn giữ trong report, biết chúng nằm ở mục nào, và chỉ đúng được file code tương ứng. Còn với các hình ở Chapter 3 và Chapter 4, điều quan trọng nhất là hiểu mỗi hình đang phục vụ luận điểm nào trong thesis: hình methodology phải chứng minh logic của pipeline; hình experiment phải chứng minh vì sao kết luận trong Chapter 4 là hợp lý.